

Lesson Notes

MODULE 3 | LESSON 3

TREES

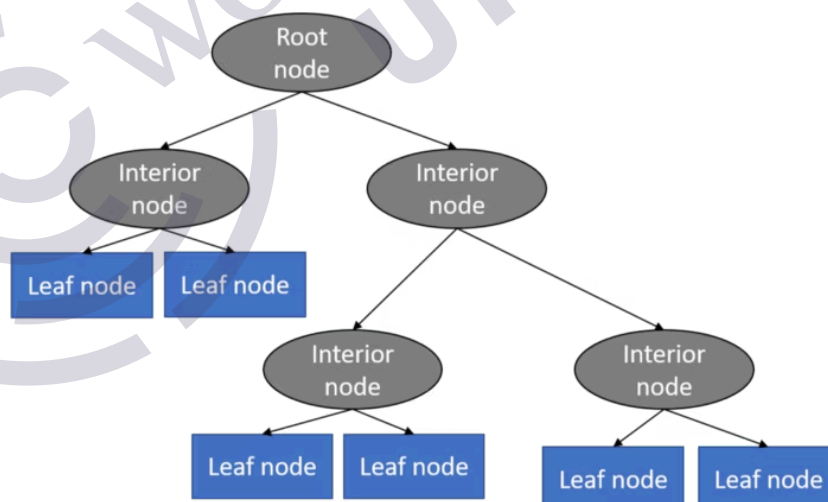
Reading Time	90 minutes
Prior Knowledge	Logistic regression
Keywords	Gini Impurity, Nodes, Homogeneous nodes, Boosting, ROC Curve

We have looked at one of the tools for classification, namely Naïve Bayes classification. This method builds on the framework of Bayesian statistics and makes use of a prior and likelihood distribution. These distributions are based on the data we train the model on, and we make the naïve assumption of independence between predictors. Oftentimes, this is not the case with real-world problems. Let us now look at an alternative method that does not make such an assumption and is widely used in practice.

1. Classification Trees

To understand tree-based models, we need to understand the basic building blocks, which are decision trees. The components of decision trees are shown in figure 1. These are nodes that have specific names based on their position in the flow diagram. By following the flow from top to bottom, we initially have the root, interior, and finally the leaf nodes. The splits into left/right directions are called **branches**.

Figure 1: The Basic Components and Structure of a Decision Tree.



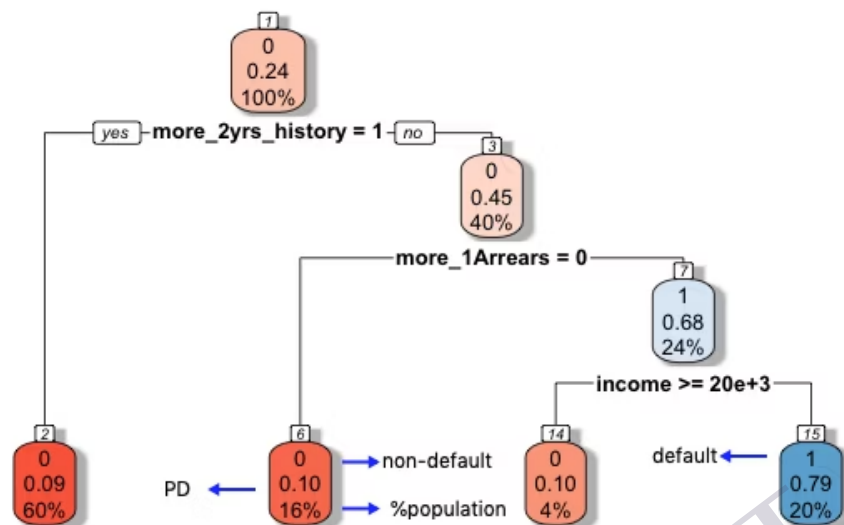
Source: python-course.edu, Decision trees in python, June 2022. <https://python-course.eu/machine-learning/decision-trees-in-python.php>.)

To conceptualize the application of the decision tree, we'll look at a credit risk default example for a bank loan based on just three explanatory variables. The target variable classes are 0: non-default, 1: default. Keep in mind that this example is to convey the idea of a decision tree as the banking industry uses much more explanatory variables/predictors in practice.

The first of the three predictors is a binary variable that flags an applicant if they have more than two years of credit history in the credit bureau. This would be valuable to a bank as it allows the bank to assess how the client performs financially. The second predictor is also a binary variable such that it is 1 if the client has had more than 1 missed

payment for other loans in the last 2 years and 0 otherwise. The third predictor is the client's available monthly income. Assume the currency is in South African rand. The figure identifies the rules that would lead us to the particular target classes. For instance, figure 2 shows that applicants with more than two years of credit history have a 9% probability of defaulting (PD) by looking at the path that leads to the bottom left-most leaf node.

Figure 2: A decision tree model applied to credit risk application data on 3 predictors.



This population makes up 60% of the population. Following the path to the right, we look at those with at most two years of credit history. This population has a PD of 45%. Splitting this population further into those that had more than 1 arrears in their two-year credit history and have an income under 20,000 rand, there is a 79% chance of default. This group makes up 20% of the population and has a high chance of default. It is important to assess the logic and the sensibility of the rules as oftentimes the analyst will need to explain this to stakeholders. In this example, the data confirms our suspicion that those with lower incomes, little credit history and history of missing more than one payment are likely to default.

Now that we understand how to use decision trees, let us look at how the tree is trained on data. To do this, we first examine some commonly used metrics to determine the splitting at branches.

2. Gini Impurity

Decision trees split the samples by sequentially selecting the predictors according to a calculated measure. The measure we look at is **Gini impurity**, named after Corrado Gini. It is given by $G = 1 - \sum_k (p_k)^2$, (3.1) where the sum is over all possible $k = 1, \dots, c$ target classes. In our example, this is seen in figure 2 for $k = 1, 2$. The terms p_k are the proportion of the class in that particular node. For instance, figure 2 splits the more than two years of credit history variable first. This feature has only two classes and we have the default percentage of the classes in table 1.

Table 1

More than 2 yrs credit history	% default
YES	9%
NO	91%

We therefore have from (3.1),

$$G = 1 - [(0.09^2) + (0.91^2)] \\ = 0.164$$

The idea behind Gini impurity is to obtain as pure a node as possible, thereby minimizing the risk of misclassification. We want as low a Gini impurity as possible to achieve this. You can see that the left-most leaf node has a very low percentage of default population. Similarly, the inner node for those with ≤ 2 yrs credit history has a Gini impurity of C

Table 2

Leaf	Gini Impurity
≤ 2yrs history, more than 1 arrears, at least R20k income	18%
≤ 2yrs history, more than 1 arrears, and < R20k income	33%

The variable at the root node is chosen to produce the purest node, i.e., the lowest Gini impurity. The decision tree has to therefore split on all predictors to determine this.

Another commonly used measure for splitting a tree is entropy. Entropy or Gini impurity is used to maximize **information gain**. The reader is referred to Hull (133) should they be interested in understanding this. We've seen trees in a classification problem, but keep in mind that they can also be used for a continuous target variable. These are regression problems, which we cover next.

For an application of decision trees with some mock data, this video should clarify the concepts discussed above:

12:55

[Access video transcript here](#)

3. Regression Trees

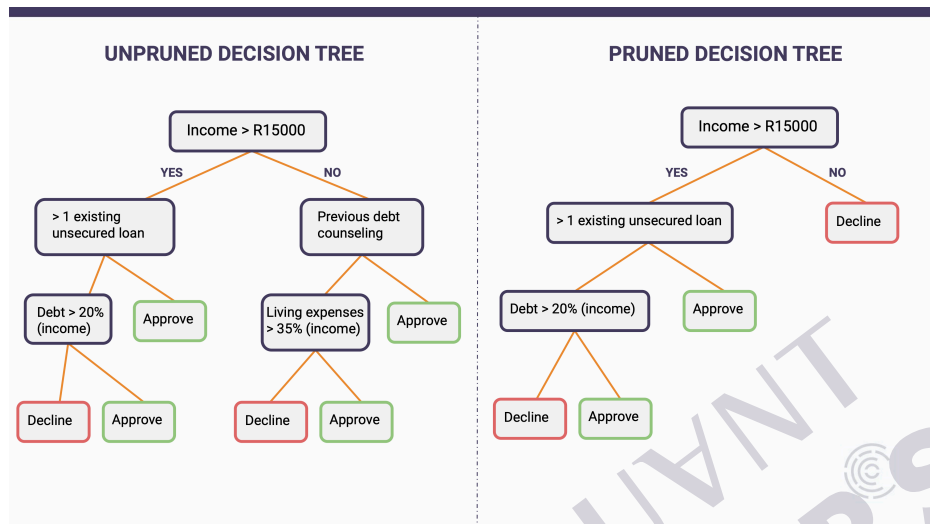
Thus far, we have referred to only classification problems; however, we may in practice be faced with a continuous target variable. In this instance, we would be interested in predicting the amount or value in a financial problem. In this type of problem, a regression tree would be applicable. The difference in the chart as compared to figure 2 is that the leaf nodes would show the mean value of the target variable. The data is split for each predictor at every point, and the split with the lowest variation between the predicted and actual target values is chosen as the splitting point. The measure of variation is the sum of squares, which should be familiar from linear regression. For any predictor i we have (3.2):

$$SSE = \sum_{i \in S_1} (y_i - \bar{y}_1)^2 + \sum_{i \in S_2} (y_i - \bar{y}_2)^2$$

with y_i the predicted value and $\bar{y}_1$ and $\bar{y}_2$ the mean value of the left and right side of the split. The split that produces the lowest SSE would be chosen. Trees, like other machine learning algorithms, are susceptible to overfitting; however, there are ways to minimize this. One of them is called pruning.

Pruning aims to reduce the size of a decision tree, thus reducing overfitting. This is done by reducing sections of the tree that are non-critical. We ideally want the tree to generalize unseen samples well. One of the pruning strategies involves producing a tree with small samples in the leaf nodes, thereby reducing the maximum depth of the tree and eliminating those overfitted leaves. This is an example of post-pruning. With pre-pruning, we specify parameters to prevent leaves with too few samples; an example would be maximum depth or minimum Gini. This early stoppage can be efficient as there are fewer computations. Figure 3 illustrates an unpruned vs. pruned tree.

Figure 3: Unpruned vs. Pruned Tree



One of the most simplistic predictive classification algorithms is logistic regression, and it is worth mentioning the comparisons to decision trees.

4. Logistic Regression vs. Decision Trees

Both methods deal with classification problems and are easily interpreted. Decision trees can split the data non-linearly; hence, the data does not have to be linearly separable.

Figure 4: Non-Linearly Separable Data

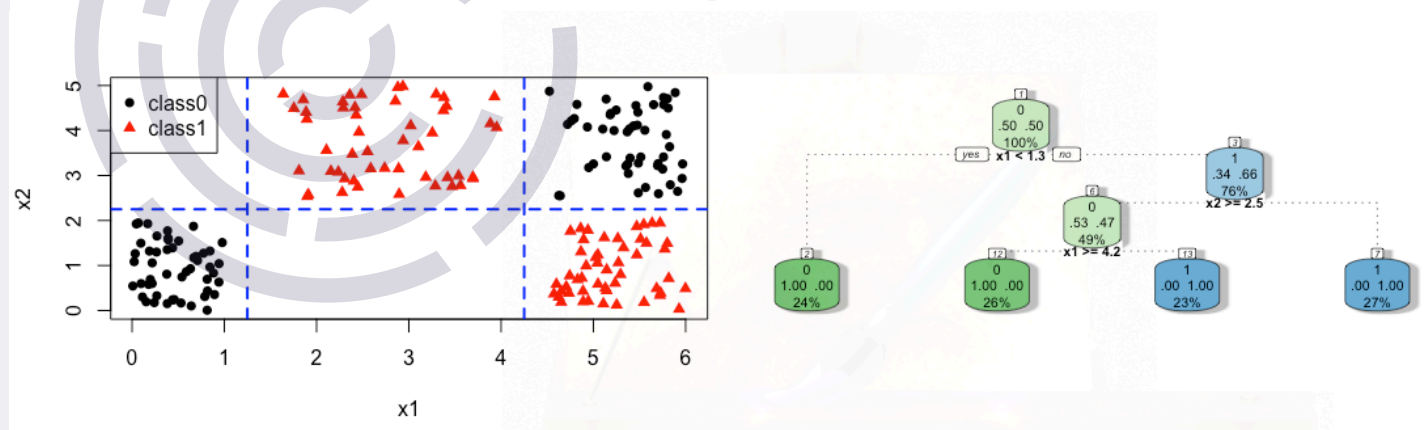


Figure 4 illustrates the non-linearly separable data of which a decision tree is more effective at class predicting. Logistic regression handles linearly separable data well. Decision trees split predictors sequentially; therefore, it is also not necessary to scale the data, unlike logistic regression. Logistic regression is also sensitive to outliers and skewness of data, which decision trees are not. With few variables in decision trees, the variable interaction is easily interpretable and shown. Quite often, a decision tree model is chosen over the logistic regression model; however, some financial institutions are regulated to use logistic regression models.

In the next section, we will look at a modified tree-based algorithm that leverages other trees to improve predictive performance.

5. Boosting

Boosting is a tree-based method that combines the power of other trees, specifically weak-learning trees, which we refer to as **weak learners**. These weak learners basically have a root node and a split on that predictor. The process combines these weak learners in a sequential manner, such that each weak learner improves upon the previous learner, meaning the final model is improved along the way. It is because of this sequential nature that the training takes longer than other parallel methods. Suppose at iteration i we have a set of predictions $P_i(X)$ from a weak learner based on a set of predictors X . The predictions at iterations can be related as:

$$P_{i+1}(X) = P_i(X) + T_i(X)$$

where $T_i(X)$ is a tree-based learner introduced at iteration i to improve on $P_i(X)$. Residuals are examined to determine adjustments to the tuning parameters of T . These residuals can be expressed as,

$$\epsilon = Y - (P_i(X) + T_i(X))$$

A popular gradient boosting algorithm used is Xgboost. These algorithms tend to easily overfit and are sensitive to noise in the data. Results are also very sensitive to parameter tuning. For a consolidation of the above concepts with mock data, watch the following video:



Thus far, we have looked at accuracy for assessing model performance. In the next section, we will examine a commonly used metric and diagram to assess model performance.

6. Receiver Operating Curve

We'll discuss a measure used for classification models, specifically the receiver operating curve (ROC) and area under the curve (AUC). These measures actually stem from the Gini coefficient and Lorentz curve, which were initially used to show income inequality in countries. We will not explore the Gini coefficient as we will stick to just the machine learning application of it. However, should the reader be interested in the Gini coefficient, we refer to Gini (1). We first show the confusion matrix for a binary classifier in table 3, which is basically a cross-tabulation of the predicted classes vs. the actual classes:

	Actual (0)	Actual (1)
Predicted (0)	TN	FN
Predicted (1)	FP	TP

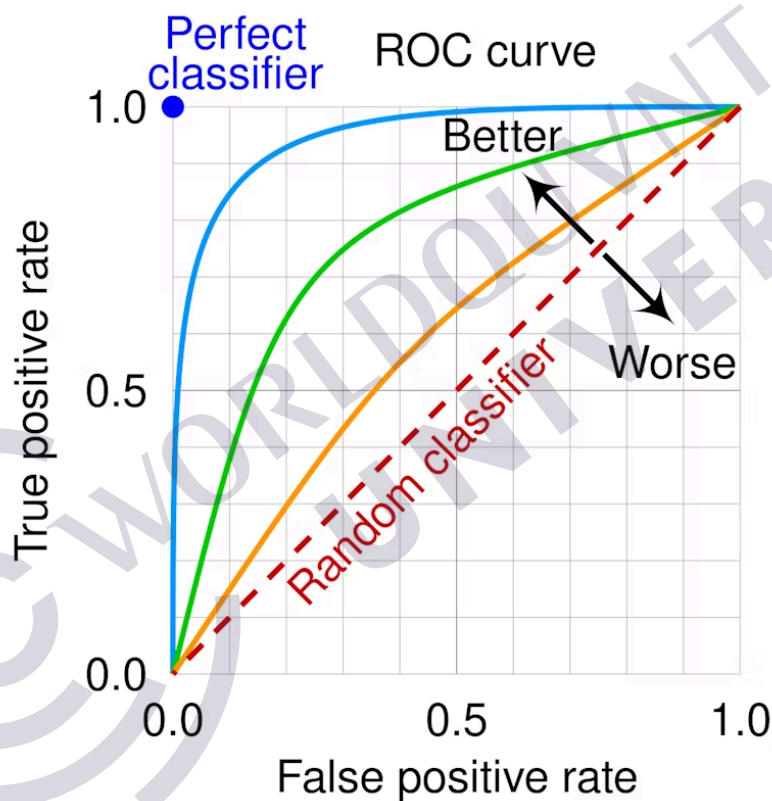
The components are **TN**, the true negatives, i.e., the correctly predicted negatives or class 0, whereas **FN** are the false negatives, i.e., the actual class 1s that are predicted as 0. Similarly, **TP** and **FP** are the true positives and false positives, respectively. Using the entries from the confusion matrix, we have the true positive (**TPR**) and false positive rates (**FPR**),

$$TPR = \frac{TP}{TP + FN}$$

$$FPR = \frac{FP}{FP + TN}$$

Keep in mind that there is a threshold of probability that assigns a prediction to a class. As an example, should 50% be the threshold, any prediction with over 50% will be assigned class 1. Thus, **FPR** and **TPR** are sensitive to the threshold.

Figure 5: ROC Curve for Different Models



Source: CMG Lee, MartinThoma, <https://commons.wikimedia.org/w/index.php?curid=109730045>

The ROC is a plot of **TPR** against **FPR** for different thresholds and is shown in figure 5. Each line or series in the chart represents a model. The dashed red line is that of a model that is a random guess, whereas the blue line is that of an ideal model that predicts each class nearly perfectly, i.e., the off-diagonal elements of the confusion matrix are close to zero. We therefore want a model as close to the blue curve as possible. This is why the green curve represents a model that is better than the model represented by the orange curve. ROC enables us to visually see which model is better. However, a quantity that can be used to compare models or determine performance is the AUC.

7. Area Under the Curve (AUC)

The AUC is simply the area between the curve and the x-axis. The higher the AUC, the better the model at differentiating classes. You may still come across the Gini coefficient being used to measure a model's performance. However, keep in mind that the AUC and Gini are linearly related and synonymous since the relation established by Schechtman and Shapton (2) is,

$$Gini = 2 * AUC - 1.$$

For a more detailed explanation of the ROC curve with an example, watch the following video:

08:44

[Access video transcript here](#)

8. Conclusion

In this lesson, we looked at tree-based algorithms in classification and regression problems. We examined the methodology behind them and looked at boosting, which combines the power of many weak learners. A modified gradient boosting tree-based algorithm was also discussed as a metric for assessing model performance. In the next lesson, we apply tree-based algorithms to a financial problem.

References

- Gini, Corrado. "On the Measurement of Concentration and Variability of Characters." *Metron - International Journal of Statistics*, vol. 63, no. 1, 2005, pp. 1–38, <https://econpapers.repec.org/article/mtnancoec/0501.htm>. Originally published in Italian in *Atti del Reale Istituto Veneto di Scienze, Lettere ed Arti*, 1914.
- Hull, Isaiah. *Machine Learning for Economics and Finance in Tensorflow 2*. Apress, 2021. https://link.springer.com/chapter/10.1007/978-1-4842-6373-0_4.
- Schechtman, Edna, and Gideon Schechtman. (2016). "The Relationship between Gini Methodology and the ROC Curve." *METRON*, vol. 77, 2019, pp. 171–178, https://www.weizmann.ac.il/math/gideon/sites/math.gideon/files/uploads/AUC_Metron%20May%202019%20v4.pdf.